Business Profile Security & Compliance Guide

Overview

This document outlines security measures, data protection, and compliance requirements for the FlowTrack Business Profile system. Given that this module handles sensitive business and financial information, strict security controls are mandatory.

Data Classification

Critical Sensitive Data (PII/Financial)

Encryption Required

- PAN Numbers (Permanent Account Number)
- Bank Account Numbers
- CIN Numbers (Corporate Identity Number)
- Udyam Registration Numbers

Encryption Status: ☑ Implemented using `SensitiveDataEncryptionConverter`

Storage:

- Database: Encrypted varchar(500)
- Memory: Decrypted only during processing
- Logs: **NEVER** log decrypted values

Sensitive Business Data

Protected but Not Encrypted

- GSTIN (GST Identification Number)
- Legal Business Names
- Registered Addresses
- Email Addresses
- Phone Numbers
- Director/Partner Names

Protection: Access control via user_id scoping

Public/Non-Sensitive Data

- Business Types (PROPRIETORSHIP, LLP, etc.)
- Verification Status
- State/City Information
- Created/Updated Timestamps

Encryption Architecture

Current Implementation

Encryption Converter

```
@Converter
public class SensitiveDataEncryptionConverter
    implements AttributeConverter<String, String>
```

Algorithm: AES (Advanced Encryption Standard)
Key Size: 256-bit (recommended)
Mode: AES/ECB (currently) ⚠
Encoding: Base64

Fields Using Encryption

1. `BusinessProfile.panNumber` → `business_profiles.pan_number`
2. `BankDetails.accountNumber` → `bank_details.account_number`

Current Limitations ⚠

1. Key Management

```
# Current (Development Only)
ENCRYPTION_SECRET_KEY=your-32-character-secret-key

# ⚠ NEVER commit actual keys to version control
# ⚠ Use environment variables or secrets manager
```

Production Requirements:

- Use AWS KMS, Azure Key Vault, or HashiCorp Vault
- Implement key rotation policy
- Use different keys per environment (dev/staging/prod)
- Enable audit logging for key access

2. Encryption Mode

Current: AES/ECB (Electronic Codebook)
Issue: Same plaintext → same ciphertext (deterministic)

Production Recommendation: Use AES/GCM (Galois/Counter Mode)

- Provides authentication
- Randomized initialization vectors (IV)
- Different ciphertext for same plaintext

Migration Path:

```java
// Updated converter for production
private static final String ALGORITHM = "AES/GCM/NoPadding";

public String convertToDatabaseColumn(String attribute) {
    SecureRandom random = new SecureRandom();
    byte[] iv = new byte[12]; // GCM standard IV size
    random.nextBytes(iv);

    GCMParameterSpec parameterSpec = new GCMParameterSpec(128, iv);
    Cipher cipher = Cipher.getInstance(ALGORITHM);
    cipher.init(Cipher.ENCRYPT_MODE, secretKey, parameterSpec);

    byte[] encrypted = cipher.doFinal(attribute.getBytes(StandardCharsets.UTF_8));

    // Prepend IV to ciphertext
    byte[] result = new byte[iv.length + encrypted.length];
    System.arraycopy(iv, 0, result, 0, iv.length);
    System.arraycopy(encrypted, 0, result, iv.length, encrypted.length);

    return Base64.getEncoder().encodeToString(result);
}
```

3. Database Encryption

Additional Layer: Consider MySQL Transparent Data Encryption (TDE)

```cpp
-- Enable for IMS database
ALTER INSTANCE ROTATE INNODB MASTER KEY;
ALTER DATABASE ims ENCRYPTION = 'Y';
```

Access Control

Multi-Tenancy Enforcement

Architectural Pattern

All data access **MUST** be scoped by:

1. **user_id**: User owns the business profile
2. **business_profile_id**: Active business context

Session Context

```java
@Component
@Scope(scopeName = WebApplicationScope.SCOPE_SESSION)
public class ActiveBusinessProfileContext {
    private Long activeBusinessProfileId;
```

```
    private Long userId;

    // CRITICAL: Validate on every request
    public void validateUser(Long requestUserId) {
        if (!requestUserId.equals(this.userId)) {
            throw new SecurityException("Tenant isolation violation");
        }
    }
}
```

Repository Security Patterns

☑ **CORRECT - Always Scope by User**:

```
Optional<BusinessProfile> findByIdAndUserId(Long id, Long userId);

List<BankDetails> findByBusinessProfileIdAndBusinessProfile_UserId(
    Long businessProfileId, Long userId
);
```

❌ **INCORRECT - Global Access**:

```
// Never use these without tenant scoping!
Optional<BusinessProfile> findById(Long id);
List<BusinessProfile> findAll();
```

Service Layer Security

Pre-Authorization:

```
@Service
public class BusinessProfileService {

    @Autowired
    private ActiveBusinessProfileContext context;

    public BusinessProfile getBusinessProfile(Long profileId, Long userId) {
        // Validate tenant
        context.validateUser(userId);

        // Scoped query
        return repository.findByIdAndUserId(profileId, userId)
            .orElseThrow(() -> new AccessDeniedException("Profile not
accessible"));
    }
}
```

Controller Layer Security

Spring Security Integration:

```
@RestController
@RequestMapping("/api/business")
public class BusinessProfileController {

    @GetMapping("/{id}")
    @PreAuthorize("isAuthenticated()")
    public ResponseEntity<BusinessProfile> getProfile(
        @PathVariable Long id,
        @AuthenticationPrincipal CustomUserDetails userDetails
    ) {
        // User ID from authentication
        Long userId = userDetails.getUserId();

        // Service enforces scoping
        BusinessProfile profile = service.getBusinessProfile(id, userId);

        return ResponseEntity.ok(profile);
    }
}
```

Role-Based Access Control (RBAC)

Roles for Business Profiles

```
public enum BusinessRole {
    OWNER,           // Full access, can delete business
    ADMIN,           // Manage details, add bank accounts
    FINANCE,         // View bank details, initiate verifications
    OPERATIONS,      // View business details only
    AUDITOR          // Read-only access including verification logs
}
```

Permission Matrix

Action	OWNER	ADMIN	FINANCE	OPERATIONS	AUDITOR
Create Business	☑	✕	✕	✕	✕
Update Business Details	☑	☑	✕	✕	✕
Delete Business	☑	✕	✕	✕	✕
Add Bank Account	☑	☑	☑	✕	✕

Action	OWNER	ADMIN	FINANCE	OPERATIONS	AUDITOR
Set Primary Bank	☑	☑	☑	✕	✕
Initiate Verification	☑	☑	☑	✕	✕
View Bank Details	☑	☑	☑	✕	☑
View Business Details	☑	☑	☑	☑	☑
View Verification Logs	☑	☑	☑	✕	☑

Implementation:

```
// New table: business_profile_roles
CREATE TABLE business_profile_roles (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    business_profile_id BIGINT NOT NULL,
    user_id BIGINT NOT NULL,
    role VARCHAR(50) NOT NULL,
    granted_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    granted_by BIGINT,
    FOREIGN KEY (business_profile_id) REFERENCES business_profiles(id) ON DELETE
CASCADE,
    FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE,
    UNIQUE KEY UK_user_business_role (user_id, business_profile_id)
);
```

Audit Logging

What to Log

1. Business Profile Events

```
// Log format
{
    "event_type": "BUSINESS_PROFILE_CREATED",
    "user_id": 123,
    "business_profile_id": 456,
    "timestamp": "2026-02-25T14:30:00Z",
    "ip_address": "192.168.1.1",
    "user_agent": "Mozilla/5.0...",
    "changes": {
        "legal_business_name": "New Business Pvt Ltd",
        "business_type": "PRIVATE_LIMITED"
    }
}
```

Events to Log:

- BUSINESS_PROFILE_CREATED
- BUSINESS_PROFILE_UPDATED (with diff)
- BUSINESS_PROFILE_DELETED
- VERIFICATION_STATUS_CHANGED
- ONBOARDING_STAGE_CHANGED

2. Bank Account Events

```
{
    "event_type": "BANK_ACCOUNT_ADDED",
    "user_id": 123,
    "business_profile_id": 456,
    "bank_details_id": 789,
    "bank_name": "State Bank of India",
    "is_primary": true,
    "masked_account": "****1234"  // ⚠ Never log full account number
}
```

Events to Log:

- BANK_ACCOUNT_ADDED
- PRIMARY_BANK_CHANGED
- BANK_VERIFICATION_INITIATED
- BANK_VERIFICATION_SUCCESS
- BANK_VERIFICATION_FAILED

3. Verification Events

Already captured in `verification_logs` table with:

- IP address
- User agent
- External reference IDs
- Request/response payloads (sanitized)

Audit Log Implementation

```
@Service
public class AuditService {

    public void logBusinessEvent(
        String eventType,
        Long userId,
        Long businessProfileId,
        String ipAddress,
        Map<String, Object> changes
    ) {
        AuditLog log = AuditLog.builder()
```

```
            .eventType(eventType)
            .userId(userId)
            .businessProfileId(businessProfileId)
            .ipAddress(ipAddress)
            .timestamp(Instant.now())
            .changes(sanitizeForLogging(changes))  // Remove sensitive data
            .build();

        auditRepository.save(log);

        // Optional: Send to external SIEM
        siem.send(log);
    }

    private Map<String, Object> sanitizeForLogging(Map<String, Object> data) {
        // Never log: PAN, account numbers, passwords
        data.remove("pan_number");
        data.remove("account_number");

        // Mask: Email, phone
        if (data.containsKey("email")) {
            data.put("email", maskEmail((String) data.get("email")));
        }

        return data;
    }
}
```

Compliance Requirements

Indian Regulatory Landscape

1. Information Technology Act, 2000 (IT Act)

Section 43A: Privacy obligations for sensitive personal data

- **Applies to**: PAN, bank account numbers
- **Requirement**: Implement reasonable security practices
- **Penalty**: Up to ₹5 crore for data breach

Compliance Checklist:

- ☑ Encryption of sensitive data
- ☑ Access control mechanisms
- ☑ Audit logging
- ⚠ TODO: Incident response plan
- ⚠ TODO: Data breach notification procedure

2. Digital Personal Data Protection Act, 2023 (DPDP)

Data Principal Rights:

- Right to access personal data
- Right to correction
- Right to erasure ("right to be forgotten")
- Right to nominate (in case of death/incapacity)

Implementation Requirements:

```
// User data access endpoint
@GetMapping("/api/user/data-export")
public ResponseEntity<UserDataExport> exportUserData(
    @AuthenticationPrincipal CustomUserDetails user
) {
    // Export all user's business profiles, bank details, verification logs
    return ResponseEntity.ok(dataExportService.exportForUser(user.getId()));
}

// User data deletion endpoint
@DeleteMapping("/api/user/delete-account")
public ResponseEntity<Void> deleteUserAccount(
    @AuthenticationPrincipal CustomUserDetails user
) {
    // Cascade delete: business_profiles → bank_details, verification_logs
    userService.deleteCompleteUserData(user.getId());
    return ResponseEntity.noContent().build();
}
```

3. GST Compliance

GSTIN Validation:

- Must validate format: 29ABCDE1234F1Z5
- Should verify with GSTN portal
- Log verification attempts for audit

4. PAN Validation

Source: NSDL or Protean (formerly NSDL)

- Must validate format: ABCDE1234F
- Should verify against Income Tax Department database
- Never store/display full PAN in logs

5. Banking Regulations

Penny Drop Verification:

- Deposit ₹1 to verify account ownership
- Matches account holder name
- Required before enabling settlements

Data Retention Policy

Retention Periods

```yaml
business_profiles:
  active: Indefinite
  deleted: 7 years (for tax purposes)

bank_details:
  active: Indefinite
  after_business_deletion: 7 years

verification_logs:
  all: 7 years (regulatory compliance)
  archival: After 1 year, move to cold storage
```

Implementation

```java
@Scheduled(cron = "0 0 2 * * *")  // Daily at 2 AM
public void archiveOldVerificationLogs() {
    LocalDateTime archiveThreshold = LocalDateTime.now().minusYears(1);

    List<VerificationLog> oldLogs =
repository.findByCreatedAtBefore(archiveThreshold);

    // Move to archive table
    verificationLogArchiveRepository.saveAll(oldLogs);

    // Delete from main table
    repository.deleteAll(oldLogs);

    log.info("Archived {} verification logs", oldLogs.size());
}

@Scheduled(cron = "0 0 3 1 * *")  // Monthly on 1st at 3 AM
public void purgeOldArchivedLogs() {
    LocalDateTime purgeThreshold = LocalDateTime.now().minusYears(7);

    int deleted =
verificationLogArchiveRepository.deleteByCreatedAtBefore(purgeThreshold);

    log.info("Purged {} archived logs (>7 years old)", deleted);
}
```

Security Best Practices

1. Input Validation

Server-Side Validation (Mandatory)

```
@Entity
public class BusinessProfile {

    @Pattern(
        regexp = "^[0-9]{2}[A-Z]{5}[0-9]{4}[A-Z]{1}[1-9A-Z]{1}Z[0-9A-Z]{1}$",
        message = "Invalid GSTIN format"
    )
    private String gstin;

    @Pattern(
        regexp = "^[A-Z]{5}[0-9]{4}[A-Z]{1}$",
        message = "Invalid PAN format"
    )
    private String panNumber;

    @Size(min = 11, max = 11, message = "IFSC must be 11 characters")
    private String ifscCode;
}
```

Client-Side Validation (User Experience)

- JavaScript validation for immediate feedback
- Never rely solely on client-side validation
- Always revalidate on server

2. SQL Injection Prevention

☑ **SAFE - Using JPA/JPQL**:

```
// Parameterized queries prevent SQL injection
@Query("SELECT bp FROM BusinessProfile bp WHERE bp.gstin = :gstin")
Optional<BusinessProfile> findByGstin(@Param("gstin") String gstin);
```

✕ **DANGEROUS - Raw SQL with concatenation**:

```
// NEVER DO THIS
String sql = "SELECT * FROM business_profiles WHERE gstin = '" + gstin + "'";
```

3. Cross-Site Scripting (XSS) Prevention

Thymeleaf Auto-Escaping:

```
<!-- SAFE - Thymeleaf escapes by default -->
<span th:text="${businessProfile.legalBusinessName}"></span>

<!-- DANGEROUS - Unescaped -->
<span th:utext="${businessProfile.legalBusinessName}"></span>
```

REST API Response:

```
// Spring automatically escapes JSON responses
@GetMapping("/api/business/{id}")
public ResponseEntity<BusinessProfile> getProfile(@PathVariable Long id) {
    // No manual escaping needed
    return ResponseEntity.ok(service.getProfile(id));
}
```

4. Cross-Site Request Forgery (CSRF) Protection

Spring Security CSRF (enabled by default):

```
@Configuration
public class SecurityConfig extends WebSecurityConfigurerAdapter {

    @Override
    protected void configure(HttpSecurity http) throws Exception {
        http
            .csrf() // CSRF protection enabled
            .csrfTokenRepository(CookieCsrfTokenRepository.withHttpOnlyFalse());
    }
}
```

Thymeleaf Forms:

```
<!-- CSRF token automatically added -->
<form th:action="@{/business/create}" method="post">
    <!-- form fields -->
</form>
```

5. Rate Limiting

Prevent Brute Force/DoS:

```
@Component
public class VerificationRateLimiter {
```

```
    private final Map<String, RateLimiter> limiters = new ConcurrentHashMap<>();

    public boolean allowVerification(String gstin) {
        RateLimiter limiter = limiters.computeIfAbsent(
            gstin,
            k -> RateLimiter.create(5.0)  // 5 requests per second
        );

        return limiter.tryAcquire();
    }
}

// Usage
if (!rateLimiter.allowVerification(gstin)) {
    throw new RateLimitExceededException("Too many verification attempts");
}
```

6. Secure API Integration

External Verification APIs:

```
@Service
public class GSTVerificationService {

    @Value("${gst.api.key}")
    private String apiKey;  // From environment variable

    public VerificationResult verifyGSTIN(String gstin, String ipAddress) {
        // Use HTTPS only
        String url = "https://api.gstn.gov.in/verify/" + gstin;

        HttpHeaders headers = new HttpHeaders();
        headers.set("Authorization", "Bearer " + apiKey);
        headers.set("X-Client-IP", ipAddress);

        // Log request (sanitized)
        log.info("Verifying GSTIN: {}****{}",
            gstin.substring(0, 5),
            gstin.substring(gstin.length() - 3)
        );

        RestTemplate restTemplate = new RestTemplate();
        // ... make request, handle response
    }
}
```

Security Checklist

Pre-Production Security Audit

Encryption

- ☐ Production encryption keys configured via secrets manager
- ☐ AES/GCM mode implemented (not ECB)
- ☐ Key rotation policy documented
- ☐ Different keys per environment

Access Control

- ☐ All repository methods scope by user_id or business_profile_id
- ☐ ActiveBusinessProfileContext validates on every request
- ☐ Role-based access control implemented (if multi-user businesses)
- ☐ Spring Security @PreAuthorize annotations on sensitive endpoints

Logging & Monitoring

- ☐ Audit logs capture all CRUD operations
- ☐ Sensitive data never logged (PAN, account numbers)
- ☐ Verification attempts logged with IP/user-agent
- ☐ Alerting configured for suspicious activity

Compliance

- ☐ Data export endpoint implemented (DPDP compliance)
- ☐ Data deletion endpoint implemented (Right to be forgotten)
- ☐ Data retention policy automated
- ☐ Privacy policy updated to reflect new data collection

API Security

- ☐ CSRF protection enabled
- ☐ Rate limiting implemented on verification endpoints
- ☐ HTTPS enforced (no HTTP)
- ☐ CORS configured restrictively

Validation

- ☐ Server-side validation on all inputs
- ☐ GSTIN format validation
- ☐ PAN format validation
- ☐ IFSC format validation
- ☐ SQL injection prevention verified

Database Security

- ☐ Foreign keys with ON DELETE CASCADE
- ☐ Triggers enforce business rules (primary bank account)
- ☐ Indexes on user_id, gstin, pan_number

- ☐ Database backups encrypted
- ☐ MySQL TDE enabled (optional but recommended)

Incident Response Plan

Data Breach Response

1. Detection

- Monitor for unusual access patterns
- Alert on bulk data exports
- Track failed login attempts

2. Containment

```
# Disable affected user accounts
UPDATE users SET enabled = FALSE WHERE id IN (affected_user_ids);

# Revoke API keys
UPDATE api_keys SET revoked = TRUE WHERE user_id IN (affected_user_ids);

# Lock affected business profiles
UPDATE business_profiles
SET verification_status = 'SUSPENDED'
WHERE user_id IN (affected_user_ids);
```

3. Investigation

- Review audit logs
- Check verification_logs for external API breaches
- Analyze access patterns

4. Notification

DPDP Act Requirements:

- Notify Data Protection Board within 72 hours
- Notify affected users immediately
- Provide details: What data, when, remedial actions

5. Remediation

- Force password reset for affected users
- Rotate encryption keys
- Patch vulnerability
- Update security measures

Contact Information

```
Security Team:
  Email: security@flowtrack.com
  Phone: +91-XXX-XXX-XXXX
  On-Call: security-oncall@flowtrack.com

Data Protection Officer (DPO):
  Name: [DPO Name]
  Email: dpo@flowtrack.com

Regulatory Authorities:
  Data Protection Board: https://www.dpb.gov.in
  CERT-In: incident@cert-in.org.in
```

Document Version: 1.0
Last Reviewed: 2026-02-25
Next Review: 2026-05-25
Owner: Security Team